choosing filters

data prep

the dataset is loaded and curred. We remove adult = 0 and NAs

```
data250_apisum_00 <- read.csv("../../../../../../01.data/2023_screen_specifics_251genotypes/0
  mutate(ivip = as.factor(ivip)) %>%
  mutate(session = as.factor(session))



data250_apisum_01 <- data250_apisum_00 %>% filter(!is.na(larvae_total_number))
data250_apisum_01$ivip <- droplevels(data250_apisum_01$ivip)

data250_apisum_02<- data250_apisum_01 %>% filter(adult_alive != "0") #remove if no adult
data250_apisum_02$ivip <- droplevels(data250_apisum_02$ivip)

data250_apisum_02$ivip <-as.factor(data250_apisum_02$ivip)
```

Once we have removed the NAs and all rows for which we don't have an adult, we can still make adjustments to the dataset. We can remove outliers to ensure that our distribution approaches a normal distribution, and we can also filter out genotypes based on a threshold for the minimum number of repetitions required to keep a genotype. We have 3 methods to handle outliers: 1)keep the outliers; 2)remove manually the outliers based on biological reasons; 3) use the interquartile range method (IQR) to remove all data outside a specific range.

The goal of this script is to study how our model reacts to changes in the outlier removal method and the minimum number of repetitions required to retain a genotype.

in the folowing loop we varry two parameters, i the minimal number of reps for the genotypes and the outlier filtering method.

```r
# Initialize an empty data frame to store results
results <- data.frame(x = numeric(0), i = numeric(0), RMSE = numeric(0),
                      n_after = numeric(0), H2=numeric(0),shapiro=numeric(0),shapiro_res=nu

# Loop through different values of x
for (i in 2:9) {
  ###filter based on IQR
    # Calculate IQR and bounds for outlier removal
    IQR_value <- IQR(data250_apisum_02$larvae_total_number)
    lower_bound <- quantile(data250_apisum_02$larvae_total_number, 0.25) - 1.5 * IQR_value
    upper_bound <- quantile(data250_apisum_02$larvae_total_number, 0.75) + 1.5 * IQR_value
    filter ="IQR"
    # Filter out outliers
    data <- data250_apisum_02[data250_apisum_02$larvae_total_number >= lower_bound & data250_


    data <- data %>%
      group_by(ivip) %>%           # Group by genotype
      filter(n() >= i) %>%             # Keep only groups with i or more repetitions
      ungroup()


    data$ivip <- droplevels(data$ivip)

    NGEN<-nlevels(data$ivip)


    # Number of rows after removing outliers
    n_after <- nrow(data)


    # Fit the model
    mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)

    # Extract fitted values and residuals
    fitted_values <- fitted(mod)
    residuals_values <- residuals(mod)

    # Perform the Shapiro-Wilk test
    shapiro_result <- shapiro.test(residuals_values)

    # Print the p-value (this is your quantification of normality)
```

```
    shapiro_res <- shapiro_result$p.value

    # Perform the Shapiro-Wilk test
    shapiro_result <- shapiro.test(data$larvae_total_number)

    # Print the p-value (this is your quantification of normality)
    shapiro <- shapiro_result$p.value


    # Calculate RMSE (Root Mean Squared Error)
    RMSE <- sqrt(mean(residuals_values^2))

    # Fit the model with a fixed formula
    mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)

    # Get the variance components
    estimates <- as.data.frame(VarCorr(mod_h2))

    # Extract the relevant variance components
    genetic_variance <- estimates$vcov[1]
    residual_variance <- estimates$vcov[3]  # Adjust based on the placement of residual varia

    nbE <-nlevels(data$session)

    # Calculate heritability (h2)
    H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))



    # Store the results in the data frame
    results <- rbind(results, data.frame(filter = filter, i = i, RMSE = RMSE, n_after = n_af
}

for (i in 3:8) {
  data <- data250_apisum_02
  filter <- "no_filter"


    data <- data %>%
      group_by(ivip) %>%           # Group by genotype
      filter(n() >= i) %>%             # Keep only groups with i or more repetitions
      ungroup()
```

```r
data$ivip <- droplevels(data$ivip)

NGEN<-nlevels(data$ivip)


# Number of rows before removing outliers
n_before <- nrow(data250_apisum_02)


# Number of rows after removing outliers
n_after <- nrow(data)

# Calculate the difference in number of observations
n_diff <- n_before - n_after

# Fit the model
mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)

# Extract fitted values and residuals
fitted_values <- fitted(mod)
residuals_values <- residuals(mod)

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(residuals_values)

# Print the p-value (this is your quantification of normality)
shapiro_res <- shapiro_result$p.value

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(data$larvae_total_number)

# Print the p-value (this is your quantification of normality)
shapiro <- shapiro_result$p.value

# Calculate RMSE (Root Mean Squared Error)
RMSE <- sqrt(mean(residuals_values^2))

# Fit the model with a fixed formula
mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)

# Get the variance components
```

```
    estimates <- as.data.frame(VarCorr(mod_h2))

    # Extract the relevant variance components
    genetic_variance <- estimates$vcov[1]
    residual_variance <- estimates$vcov[3]  # Adjust based on the placement of residual varia

    nbE <-nlevels(data$session)

    # Calculate heritability (h2)
    H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))



    # Store the results in the data frame
    results <- rbind(results, data.frame(filter =filter, i = i, RMSE = RMSE, n_after = n_aft
}

for (i in 3:8) {
 ###manual filter
  filter <-"manual_filter"



data250_apisum_03<- data250_apisum_02 %>% filter(larvae_total_number != "0")

data250_apisum_03$ivip <- droplevels(data250_apisum_03$ivip)
data <- data250_apisum_03
    data <- data %>%
      group_by(ivip) %>%           # Group by genotype
      filter(n() >= i) %>%             # Keep only groups with i or more repetitions
      ungroup()

    data$ivip <- droplevels(data$ivip)

    NGEN<-nlevels(data$ivip)


    # Number of rows before removing outliers
    n_before <- nrow(data250_apisum_02)


    # Number of rows after removing outliers
```

```r
n_after <- nrow(data)

# Calculate the difference in number of observations
n_diff <- n_before - n_after

# Fit the model
mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)

# Extract fitted values and residuals
fitted_values <- fitted(mod)
residuals_values <- residuals(mod)

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(residuals_values)

# Print the p-value (this is your quantification of normality)
shapiro_res <- shapiro_result$p.value


# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(data$larvae_total_number)

# Print the p-value (this is your quantification of normality)
shapiro <- shapiro_result$p.value

# Calculate RMSE (Root Mean Squared Error)
RMSE <- sqrt(mean(residuals_values^2))

# Fit the model with a fixed formula
mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)

# Get the variance components
estimates <- as.data.frame(VarCorr(mod_h2))

# Extract the relevant variance components
genetic_variance <- estimates$vcov[1]
residual_variance <- estimates$vcov[3]  # Adjust based on the placement of residual varia

nbE <-nlevels(data$session)

# Calculate heritability (h2)
H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))
```

```
    # Store the results in the data frame
    results <- rbind(results, data.frame(filter = filter, i = i, RMSE = RMSE, n_after = n_aft
}

print(results)
```

```
          filter i     RMSE n_after        H2      shapiro  shapiro_res NGEN
1            IQR 2 6.657805    2260 0.4108407 5.820174e-07 3.813894e-06  243
2            IQR 3 6.666687    2248 0.4112436 6.649820e-07 4.567990e-06  237
3            IQR 4 6.690371    2215 0.4113008 7.202519e-07 6.179531e-06  226
4            IQR 5 6.682081    2191 0.4114434 1.052337e-06 4.305833e-06  220
5            IQR 6 6.684478    2176 0.4136635 1.174087e-06 3.925160e-06  217
6            IQR 7 6.691100    2128 0.4163125 1.538742e-06 9.974474e-06  209
7            IQR 8 6.695692    2051 0.4062488 2.121472e-06 2.534664e-06  198
8            IQR 9 6.696062    1851 0.4047060 3.141034e-06 1.104593e-05  173
9      no_filter 3 7.492743    2331 0.3716888 3.336373e-11 2.262427e-14  237
10     no_filter 4 7.519257    2301 0.3714715 4.682873e-11 3.553872e-14  227
11     no_filter 5 7.516380    2273 0.3709082 3.906445e-11 3.071967e-14  220
12     no_filter 6 7.516380    2273 0.3709082 3.906445e-11 3.071967e-14  220
13     no_filter 7 7.555582    2231 0.3690504 1.384709e-10 5.371531e-14  213
14     no_filter 8 7.568168    2154 0.3636776 5.928548e-10 3.184938e-14  202
15 manual_filter 3 7.328357    2320 0.3844173 3.176391e-11 2.268226e-12  237
16 manual_filter 4 7.353900    2290 0.3841713 4.205613e-11 3.497580e-12  227
17 manual_filter 5 7.350153    2262 0.3837297 3.918677e-11 2.875336e-12  220
18 manual_filter 6 7.350153    2262 0.3837297 3.918677e-11 2.875336e-12  220
19 manual_filter 7 7.386433    2220 0.3820155 1.447377e-10 5.260479e-12  213
20 manual_filter 8 7.375090    2136 0.3814073 8.414128e-10 5.034994e-12  201
```

```
write.csv(x = results,
          file = "filter_annalysis.csv",
          quote = F, row.names = F)
```

```
# Initialize an empty data frame to store results
results <- data.frame(x = numeric(0), i = numeric(0), RMSE = numeric(0),
                      n_after = numeric(0), H2=numeric(0),shapiro=numeric(0),shapiro_res=nu

# Loop through different values of x
for (i in 2:9) {
  ###filter based on IQR
```

```r
# Calculate IQR and bounds for outlier removal
IQR_value <- IQR(data250_apisum_02$larvae_total_number)
lower_bound <- quantile(data250_apisum_02$larvae_total_number, 0.25) - 1.5 * IQR_value
upper_bound <- quantile(data250_apisum_02$larvae_total_number, 0.75) + 1.5 * IQR_value
filter ="IQR"
# Filter out outliers
data <- data250_apisum_02[data250_apisum_02$larvae_total_number >= lower_bound & data250_


data <- data %>%
  group_by(ivip) %>%           # Group by genotype
  filter(n() >= i) %>%             # Keep only groups with i or more repetitions
  ungroup()


data$ivip <- droplevels(data$ivip)

NGEN<-nlevels(data$ivip)


# Number of rows after removing outliers
n_after <- nrow(data)


# Fit the model
mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)

# Extract fitted values and residuals
fitted_values <- fitted(mod)
residuals_values <- residuals(mod)

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(residuals_values)

# Print the p-value (this is your quantification of normality)
shapiro_res <- shapiro_result$p.value

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(data$larvae_total_number)

# Print the p-value (this is your quantification of normality)
shapiro <- shapiro_result$p.value
```

```
    # Calculate RMSE (Root Mean Squared Error)
    RMSE <- sqrt(mean(residuals_values^2))

    # Fit the model with a fixed formula
    mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)

    # Get the variance components
    estimates <- as.data.frame(VarCorr(mod_h2))

    # Extract the relevant variance components
    genetic_variance <- estimates$vcov[1]
    residual_variance <- estimates$vcov[3]  # Adjust based on the placement of residual varia

    nbE <-nlevels(data$session)

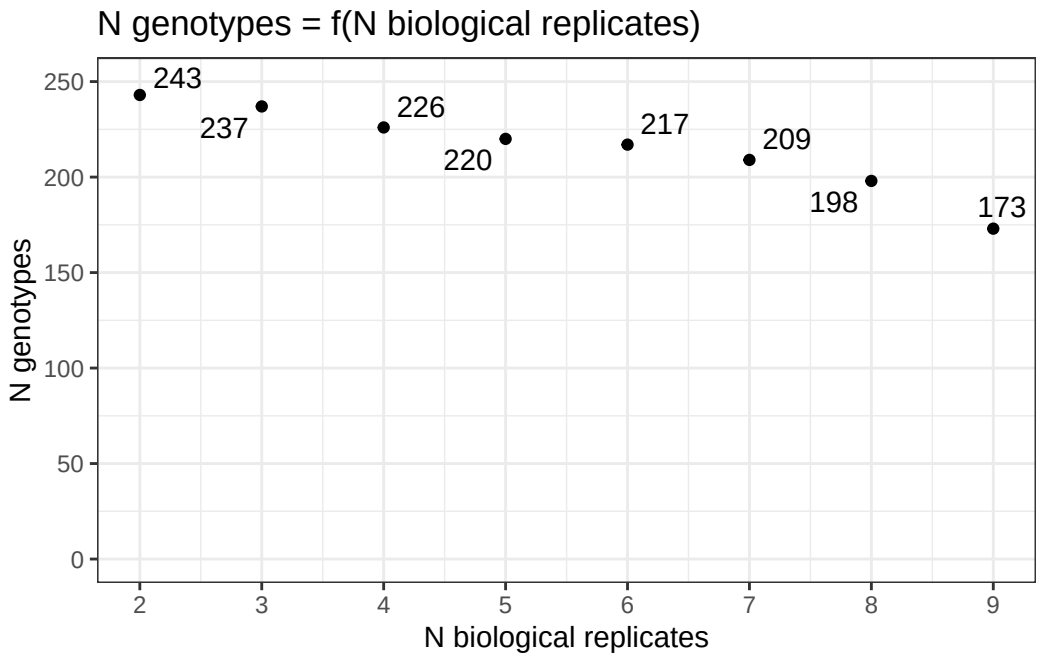
    # Calculate heritability (h2)
    H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))



    # Store the results in the data frame
    results <- rbind(results, data.frame(filter = filter, i = i, RMSE = RMSE, n_after = n_aft
}
```

We now need to visualize the data we generated to choose the best filter

```
ggplot(results, aes(x = i, y = NGEN)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,250)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "N biological replicates", y = "N genotypes") +
  ggtitle("N genotypes = f(N biological replicates)")
```

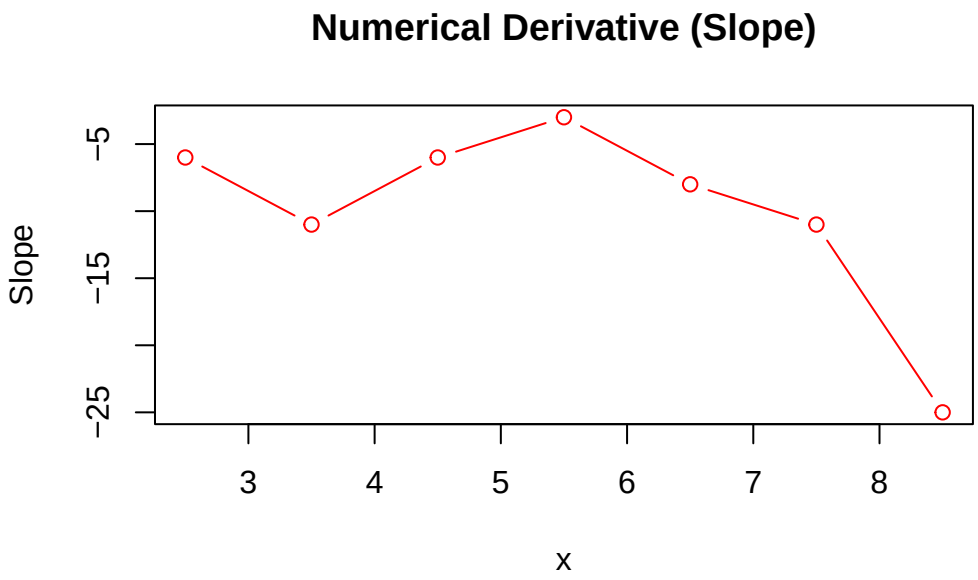



```r
y<-as.vector(results$NGEN)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```

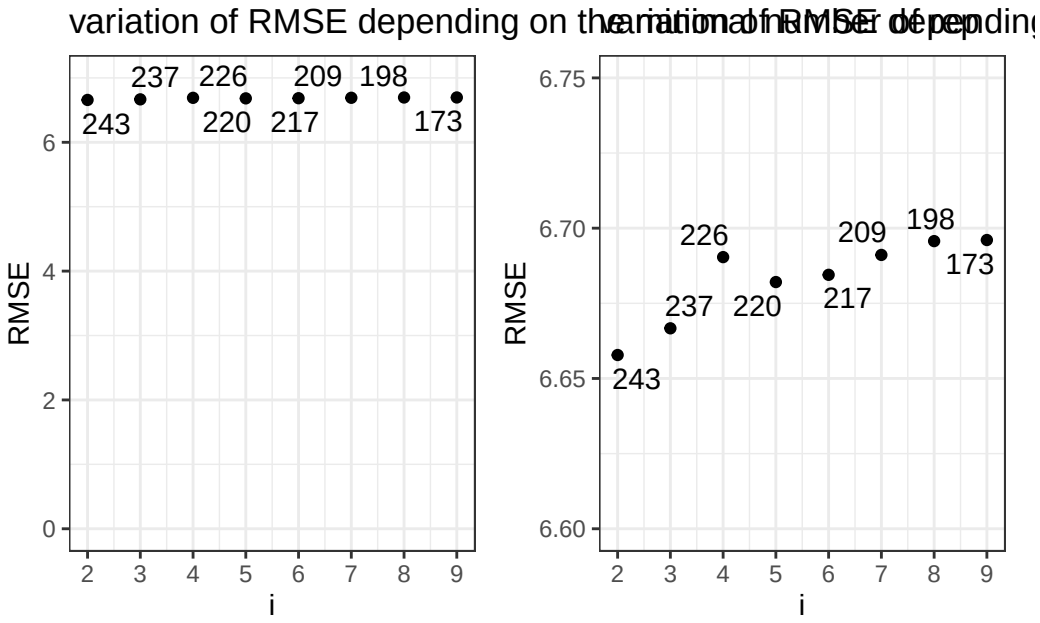


best is 6

```
away <- ggplot(results, aes(x = i, y = RMSE)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,7)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "i", y = "RMSE") +
  ggtitle("variation of RMSE depending on the minimal number of rep")

close<- ggplot(results, aes(x = i, y = RMSE)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(6.6,6.75)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "i", y = "RMSE") +
  ggtitle("variation of RMSE depending on the minimal number of rep")

away+close
```

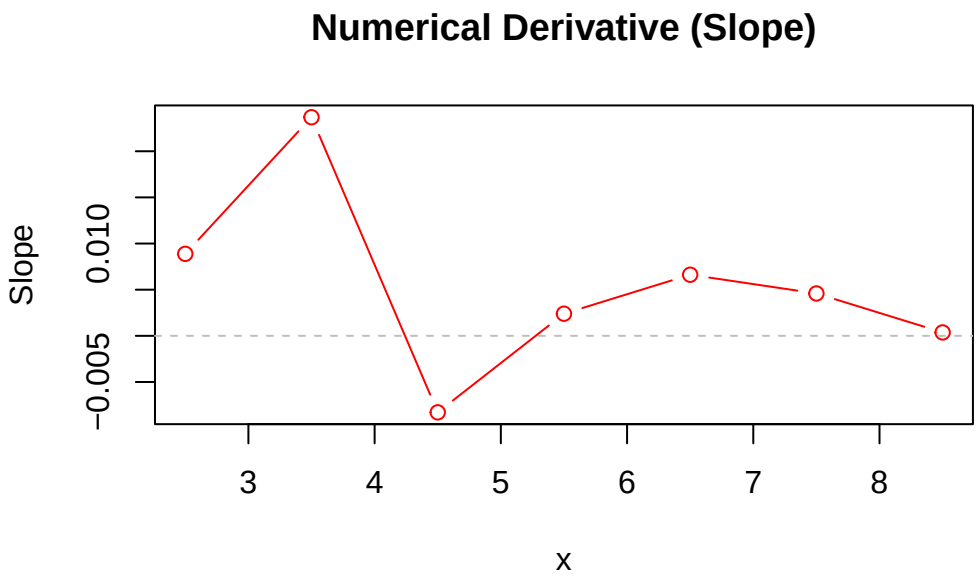



```r
y<-as.vector(results$RMSE)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

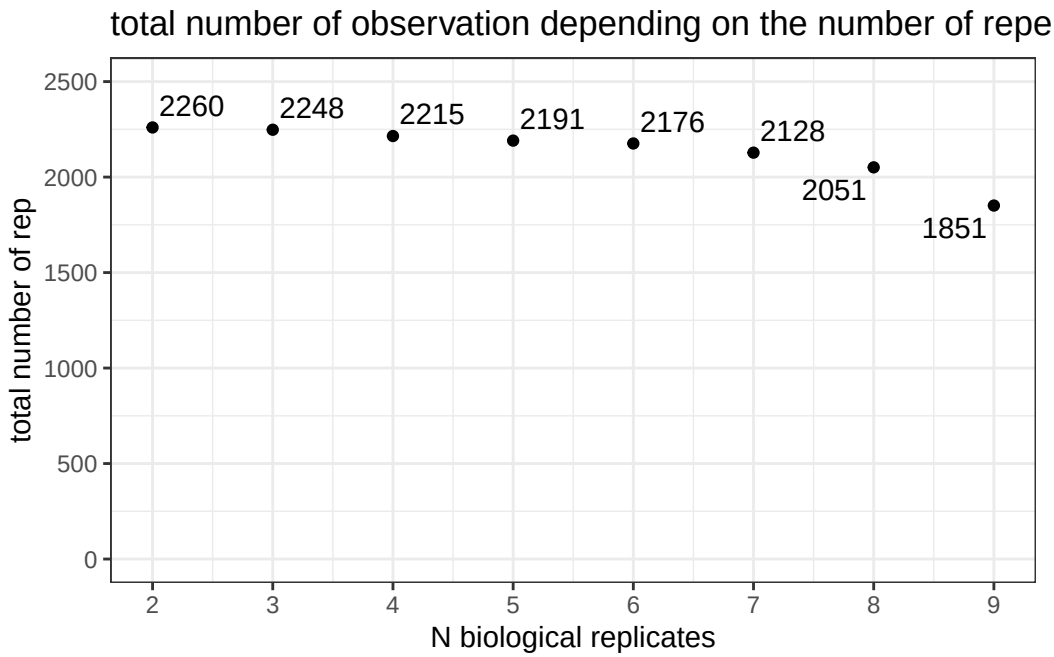
# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



not clear which filter is the best

```
ggplot(results, aes(x = i, y = n_after)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,2500)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = n_after), arrow = T) +
  labs(x = "N biological replicates", y = "total number of rep") +
  ggtitle("total number of observation depending on the number of repetitions")
```

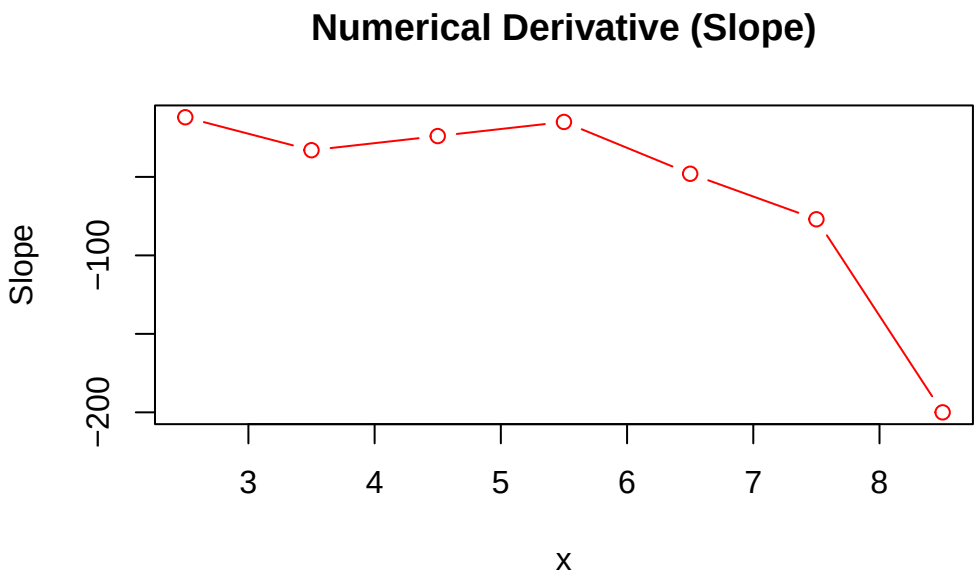



```r
y<-as.vector(results$n_after)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



best is 6

```
close<-ggplot(results, aes(x = i, y = H2)) +
  geom_point() +
  scale_x_continuous(breaks = seq(1, 13, 1)) +
  scale_y_continuous(limits = c(0.4, 0.42)) +
  theme_bw() +
  labs(
    x = "N biological replicates",
    y = "Total number of rep"
  ) +
  ggtitle("H2 depending on the number of repetitions")

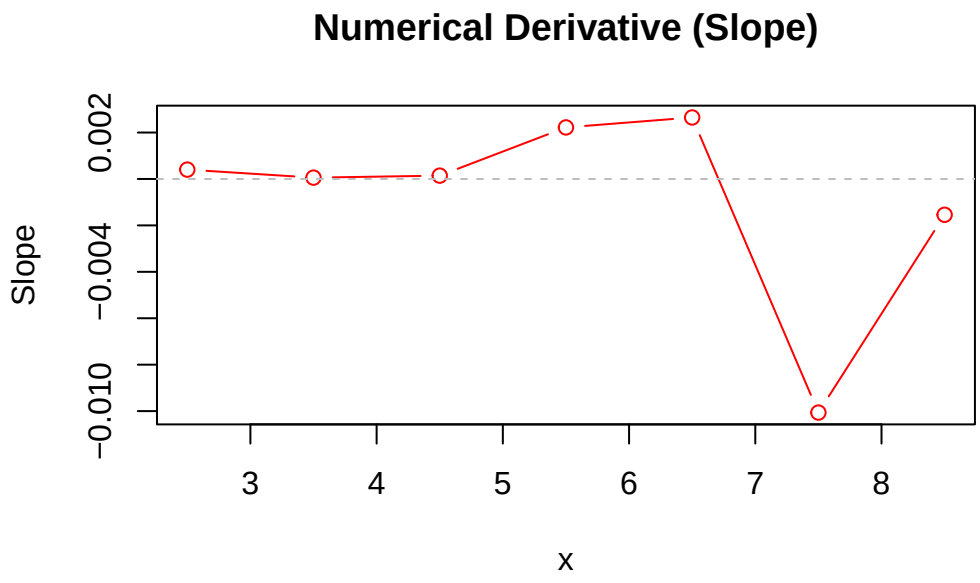
away<-ggplot(results, aes(x = i, y = H2)) +
  geom_point() +
  scale_x_continuous(breaks = seq(1, 13, 1)) +
  scale_y_continuous(limits = c(0, 0.5)) +
  theme_bw() +
  labs(
    x = "N biological replicates",
    y = "Total number of rep"
  ) +
  ggtitle("H2 depending on the number of repetitions")
```

```r
y<-as.vector(results$H2)
x<-as.vector(results$i)

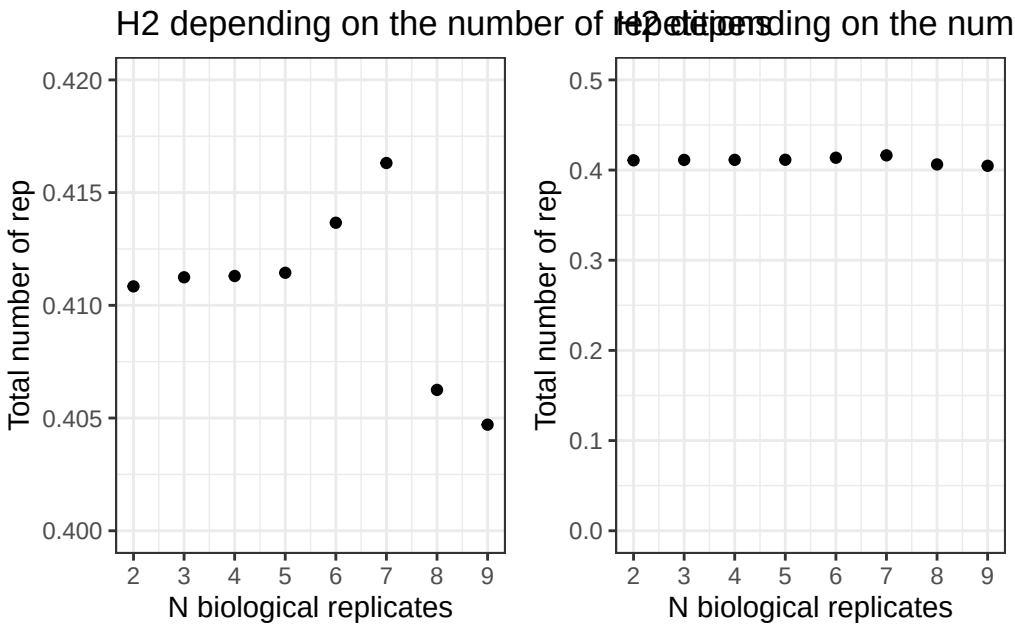
# Compute slope (finite differences)
slope <- diff(y) / diff(x)

# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```




```
close + away
```



best is 7, second best is 6